



8



13



1950 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

Line coding & Block coding

د. عماد الدين محمد

محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 22/6/2026

الاتصالات الرقمية

بسم الله الرحمن الرحيم

Course Content:

- Communication system
- Conversion techniques
- Digital data – Digital signal (line coding)
- Unipolar Encoding
- Biphasic Schemes
- Bipolar line coding
- Block coding

The purpose of a **communication system** is to transfer data from one place to another, regardless of the distance between them. However, data cannot be transmitted in its original form, so it must first be converted into a signal, since signals are what travel through wires or through the air.

Example: When you send a WhatsApp message, the text you type is the data, while the signals transmitted through the network are the signals.

There are only two types of data:

- 1) **Digital Data:** images, files, text stored inside a computer
- 2) **Analog Data:** sound and temperature

Conversion techniques

How Do We Convert Data into a Signal??

To convert data into **Digital Signal**, the process is called **Encoding**.

To convert data into **Analog Signal**, the process is called **Modulation**.

Digital data –Digital signal

If the data is already digital and we also want to transmit it as a digital signal, we use a technique called Line Coding.

Why?

Because a computer understands only 0s and 1s, while a transmission medium (such as a wire) does not understand binary digits—it carries electrical signals.

Therefore, each 0 and 1 must be translated into a specific electrical voltage level.

In other words, instead of transmitting a "1", we transmit a particular voltage level.

Instead of transmitting a "0", we transmit a different voltage level.

Example: Imagine an Arabic speaker trying to communicate with a Japanese speaker. They would not understand each other, so they need a translator. Line Coding acts as the translator between the computer's binary language and the electrical language of the transmission medium.

Line Coding

Line Coding maps binary digits into specific voltage levels.

Line Coding is not a single technique. Rather, it is a collection of different methods for representing 0s and 1s as electrical signals.

Each method was developed to solve a problem found in the previous one.

That is why, throughout the lecture, you will encounter several different Line Coding techniques. It is not because the older methods are incorrect, but because each new method is designed to overcome the limitations of the earlier ones.

Example: Think of smartphones. Every year, a new model is released—not because the previous one no longer works, but because the new model improves on the shortcomings of the older version.

Unipolar Line Coding: Uses a single voltage level (e.g., 1 = positive voltage, 0 = zero volts).

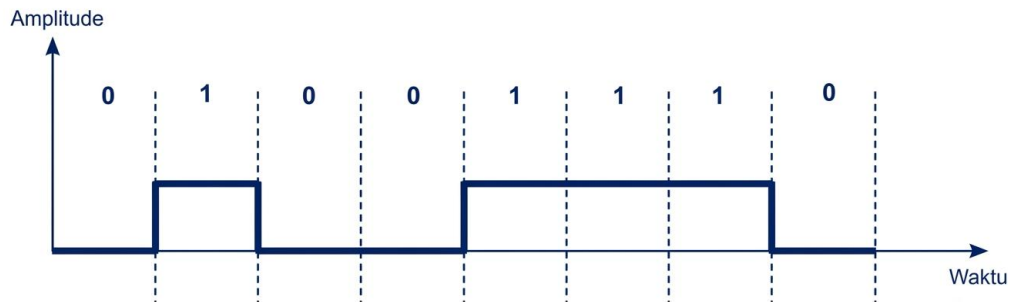
Polar Line Coding: Uses multiple voltage levels above and below zero (e.g., NRZ-L, RZ, and Biphase schemes like Manchester and Differential Manchester for self-synchronization).

Bipolar Line Coding: Uses three voltage levels: positive, negative, and zero (e.g., AMI).

Keep moving forward

Unipolar Encoding

1. UNIPOLAR LINE CODING



Unipolar pros:

Cost-Effective Implementation: The encoding and decoding hardware is highly straightforward because it only requires one power supply level and basic logic.

Low Circuit Complexity: Receivers simply need to detect the presence or absence of a threshold voltage, simplifying the overall circuit design.

threshold voltage:

The receiver does not automatically know whether a received signal represents a 0 or a 1, So, it uses a reference value called the threshold.

If the signal is above the threshold, it is interpreted as 1.

If the signal is below the threshold, it is interpreted as 0.

In other words, the threshold is simply a decision boundary used to distinguish between the two binary values.

Example: Think of passing a university course. If you score 60 or above, you pass. If you score below 60, you fail.

Here, 60 is the threshold.

Good Bandwidth Efficiency: It uses one pulse per bit, making it relatively efficient in terms of required bandwidth compared to encoding schemes.

Keep exploring

Unipolar Cons:

Poor Synchronization: If there is a **long** string of consecutive ones or zeros.

DC Bias (Component): The signal's average amplitude is non-zero, which creates a Direct Current (DC) component. This limits its use across transformers and AC-coupled mediums.

High Power Consumption: It consumes more power to send a single bit per.

Unipolar Types

- Unipolar Non-Return-to-Zero (**NRZ**)
- Unipolar Return-to-Zero (**RZ**)

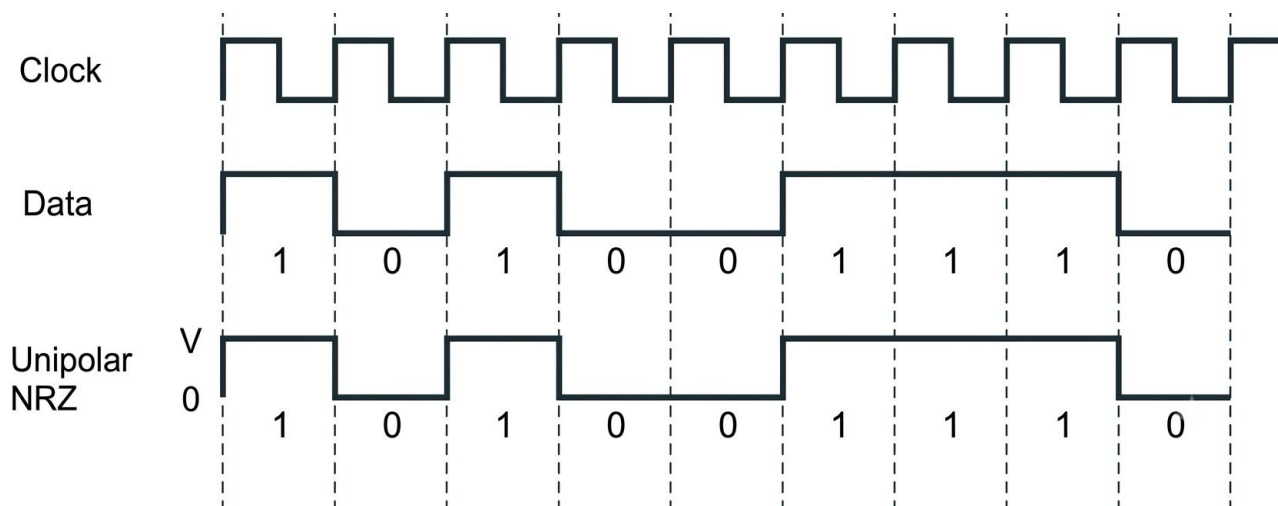
The difference between them lies in how the signal changes during the transmission of bits.

1. Unipolar Non-Return-to-Zero (NRZ)

-How it works: The signal maintains a constant, positive voltage level for the entire duration of a bit '1' and drops to zero voltage for a bit '0'. The signal does not return to zero in the middle of the bit.

-PROS: Simple to implement and requires less bandwidth.

-CONS: Contains a DC (Direct Current) component, which can damage AC-coupled circuits, and lacks built-in clock synchronization for long strings of '1's or '0's.

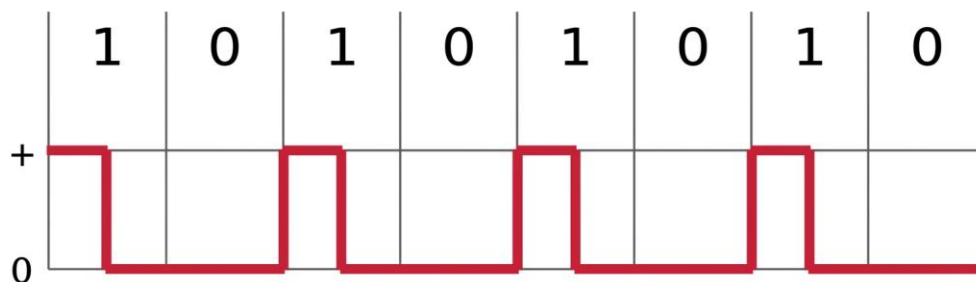


2. Unipolar Return-to-Zero (RZ)

How it works: Similar to NRZ, a bit '1' is represented by a positive voltage, but the signal **drops** back to zero volts in the **middle** of the bit duration. A bit '0' **remains** at zero voltage for the **entire** cycle.

Pros: The mid-bit transition provides a built-in timing or clock signal, making synchronization between the sender and receiver easier.

Cons: Requires twice the bandwidth of NRZ and increases system complexity because each bit requires multiple signal changes.



Unipolar RZ encoding for 10101010

Biphase scheme

A family of digital line coding techniques used in telecommunications to transmit binary data over a physical medium. Their defining feature is that they **include** a guaranteed voltage **transition** in the middle of every bit interval, which provides excellent **self-synchronization** and eliminates the DC component.

- Manchester encoding
- Differential Manchester encoding

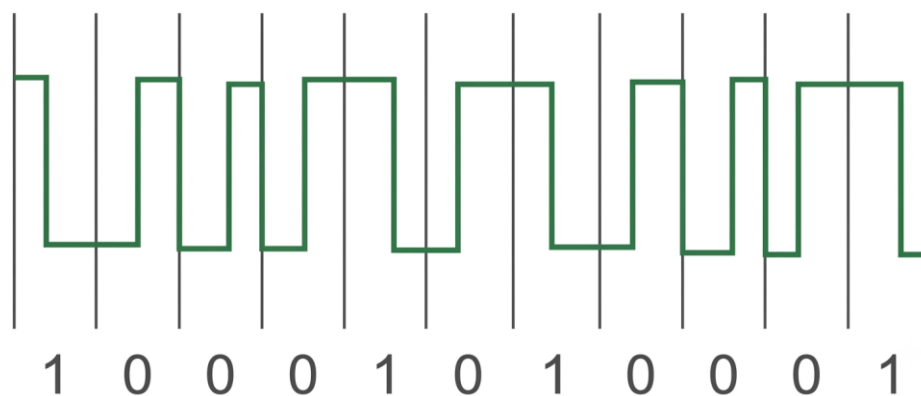
1- Manchester encoding:

Manchester encoding is a digital modulation technique that **combines** clock and data into a single, self-synchronizing stream. Instead of determining a bit's value by its voltage level alone, it uses the direction of the voltage transition (edge) at the middle of every bit.

-The IEEE 802.3 Standard: A **logic 0** is represented by a **Low-to-High** transition in the middle of the bit.

-A **logic 1** is represented by a **High-to-Low** transition in the middle of the bit.

Ad Astra — To the Stars



Manchester encoding (PROS)

- **Self-Clocking:** Because every bit has a transition in the middle of its period, the receiver can easily extract the clock signal directly from the incoming data.
- **No DC Component:** Each bit duration contains equal positive and negative voltage contributions, eliminating baseline wandering and buildup of DC current.
- **High Transition Density:** The constant stream of edges makes it exceptionally **easy to detect** the start and end of data frames and maintain signal lock.
- **Error Detection:** An error can be spotted **immediately** if the expected mid-bit transition fails to occur.

Manchester encoding (CONS)

- **High Bandwidth Requirement:** It requires up to **twice** the signal bandwidth compared to basic NRZ coding because two logic states are transmitted for every single bit of information.
- **Reduced Data Throughput:** Due to the need for mid-bit transitions, the **baud rate** (signaling rate) is twice the **bit rate**, making it inefficient for very high-speed links.
- **Poor at High Frequencies:** The signal transitions can cause complications at **higher** transmission speeds, making it unsuitable for modern high-speed data networks.

Best Use Cases

Because of its simplicity, resilience to noise, and self-clocking ability, **Manchester encoding** is still **widely** used in slower, robust, and connection-critical environments. Common applications include:

Automotive: Protocols like PSI5 for airbag sensors.

Consumer Electronics: Infrared remote control signals.

Industrial & Smart Lighting: DALI standards and Profibus PA. (sensors).

Wireless/RFID: Simple, low-power short-range communication frameworks.

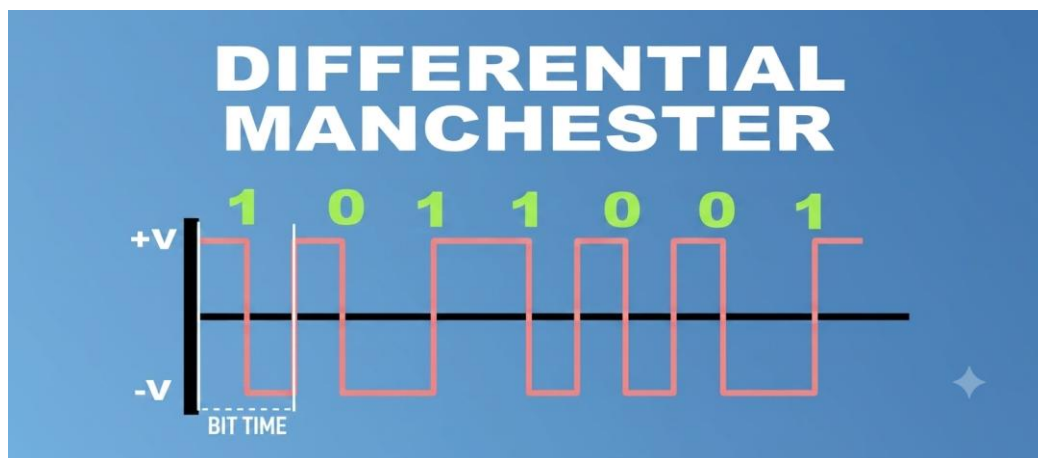
2- Differential Manchester encoding

Differential Manchester Encoding is a self-synchronizing digital line code where both **data** and **clock** signals are **combined**. Data is represented by the presence or absence of a signal transition at the start of the bit period, while a mandatory **mid-bit** transition ensures **clock** synchronization.

Binary 0: There is a transition at the beginning of the bit interval.

Binary 1: There is no transition at the beginning of the bit interval.

Mid-Bit Transition: Regardless of whether the bit is a 0 or a 1, there will **always** be a transition exactly in the middle of the bit period. This mid-bit edge acts as a **continuous timing reference** so receiver clocks do not drift.



Advantages

- **Noise Immunity:** Because data is determined by **relative transitions** rather than absolute voltage levels, the code remains reliable even if the physical wires are accidentally swapped or inverted
- **Self-Clocking:** Combining the clock and data signals removes the need for a separate clock wire, **reducing** costs and **preventing** synchronization errors.
- **DC Balancing:** The constant switching between high and low states averages out, resulting in **no direct current (DC)** component.

Drawbacks

- **High Bandwidth:** Because there is **at least** one mandatory transition per bit, the signaling rate is double that of the original data rate. This means it requires roughly twice the bandwidth of basic encoding schemes like NRZ.
- **Complexity:** Decoding hardware requires slightly more **complex** logic to compare the current edge against the previous bit's state.

Common Applications

Widely used in networking, specifically in the IEEE 802.5 Token Ring, magnetic tape readers, and older telecommunication protocols.

Bipolar Line Coding

Bipolar line coding is a signaling scheme used in telecommunications and data communications that utilizes **three** voltage levels: positive, negative, and zero. It is primarily designed to prevent baseline wandering and eliminate the DC (direct current) component in transmitted signals.

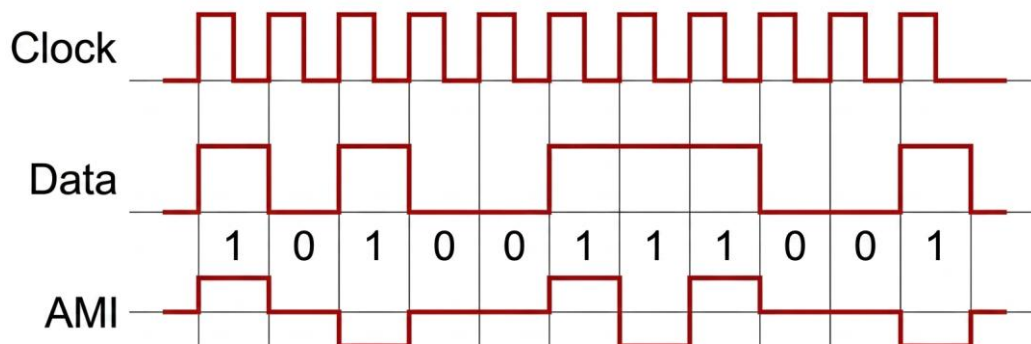
Unlike standard polar schemes that only use two voltage levels, bipolar schemes use three distinct levels: **+V**, **-V**, and **0**.

Bipolar line coding types:

- Alternate Mark Inversion (AMI)
- Pseudoternary
- Substitution Techniques (For Long Strings of Zeros)

1- AMI:

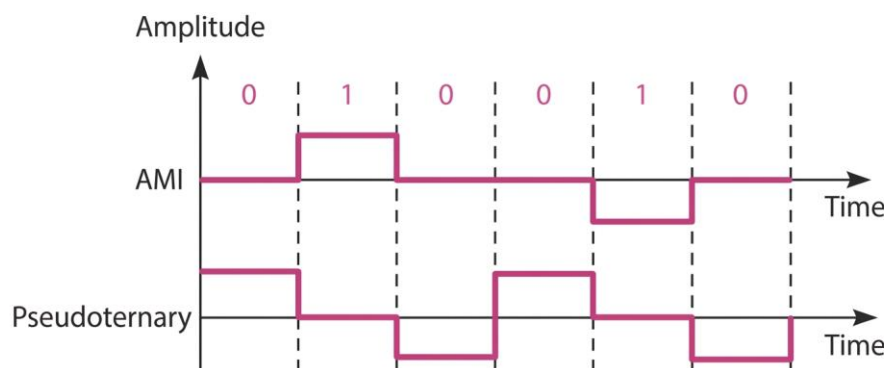
- **How it works:** A binary 0 is represented by a zero voltage. Binary 1s are represented by **alternating** positive and negative voltages.
- **PROS:** If one pulse is corrupted by noise, its polarity usually helps the receiver detect the error.
- **CONS:** Long sequences of consecutive 0s can cause **synchronization** issues, as the signal remains at a flat zero voltage.



Progress is measured not by perfection, but by the willingness to keep learning

2- Pseudoternary:

- **How it works:** This is the exact inverse of AMI. A binary 1 is represented by a zero voltage, while binary 0s are represented by alternating positive and negative voltages.
- **PROS:** Provides similar DC-balancing and noise-detection characteristics as AMI, simply reversing the bit representations.
- **CONS:** ??? (H/w)



3- Substitution Techniques (For Long Strings of Zeros)

The goal is to prevent the loss of synchronization during long streams of consecutive zeros, two specialized substitution methods are often used alongside AMI.

- Bipolar 8-Zero Substitution (B8ZS)
- High-Density Bipolar 3-Zero (HDB3)

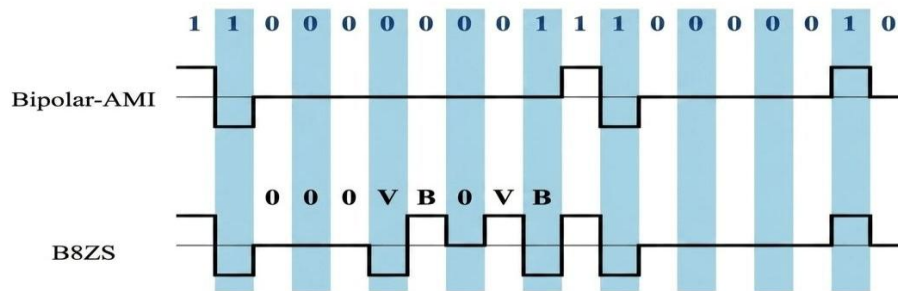
1- Bipolar 8-Zero Substitution (B8ZS):

Used primarily in North American T1 transmission systems. Whenever a sequence of eight consecutive zeros occurs, it is replaced by an intentional "bipolar violation" (a pattern containing pulses of the same polarity). The receiver recognizes this specific violation and converts it back to zeros.

Example: Imagine two people communicating using hand signals. If one person remains silent for five minutes, the other might think they have left. So, every now and then, they wave their hand even when they have no message to send, simply to indicate, "I'm still here."

This is exactly the idea behind B8ZS.

Good engineers never stop asking how and why things work



2- High-Density Bipolar 3-Zero (HDB3):

Widely used in European transmission systems. It operates similarly but only requires a sequence of **four** consecutive **zeros** to trigger a substitution and maintain synchronization.

This method is better at maintaining **synchronization**.

Why?

Because it does not leave the receiver waiting for a long period without any signal transitions or activity. This helps the receiver stay synchronized with the incoming data stream.

H/W:

Compare them with **Unipolar** and **Polar** line codes.
Explain how they perform under **noise** and **signal attenuation**.
Provide specific examples of **clock recovery mechanisms**.

Block coding

Everything we studied earlier involved **changing the shape of the signal**.

Here, however, we **modify the data itself before transmission** by adding extra bits.

Why do we add extra bits?

To make the transmission **easier**, **more reliable**, and **less prone to errors**.

Why is this necessary?

Because certain bit patterns, such as **0000** or **1111**, can cause transmission problems. Therefore, we modify the data before sending it to avoid these issues.

When the data reaches the receiver, the **extra bits are removed**, restoring the original data exactly as it was before transmission.

Example: Imagine you want to ship a **glass cup**. You wrap it with protective foam before shipping. The foam is **not part of the cup**, but it is added to protect it during transportation. Once the package arrives, the foam is removed.

The protective foam is like the **extra bits**—they are added temporarily to ensure safe transmission and removed after the data is received.

Block coding is a digital communication technique that improves signal reliability by grouping data into fixed-size blocks and adding redundant (extra) bits. These additional bits ensure sender/receiver clock synchronization and allow systems to detect—and often correct—transmission errors caused by signal noise.

The Core Process: mB/nB Coding

Block coding is usually represented as mB/nB coding, where a group of m data bits is **substituted** with a larger group of n output bits (where $n > m$). The process involves three basic steps:

Division: The incoming data stream is broken into **equal-length** blocks of m bits

Substitution: Each m -bit block is **translated** into an n -bit block using a predefined mapping/substitution table.

Combination: The newly mapped n -bit blocks are **combined** into a continuous stream to be sent to the physical layer.

Why Block Coding is Needed

Synchronization: A **long** series of consecutive 0s or 1s can cause receiver and transmitter clocks to fall out of sync. By forcing specific bit patterns, block coding **guarantees** frequent voltage transitions, keeping clocks aligned.

Error Detection and Correction: Because n is greater than m , there are **more** total possible combinations of bits than there are actual data messages. The "**unused**" combinations are reserved to detect invalid patterns or to correct errors automatically at the receiver.

Common Block Coding Schemes

4B/5B: **Converts** every 4 bits of data into a **5-bit** code. It is often combined with NRZ-I line coding to **guarantee** transitions and prevent long strings of zeros in networks like 100BASE-TX Fast Ethernet.

8B/10B: Converts an **8-bit** block into a **10-bit** pattern. Used extensively in Gigabit Ethernet and PCI Express (PCIe), it provides an exceptionally high number of unused combinations for advanced control signaling and error detection.

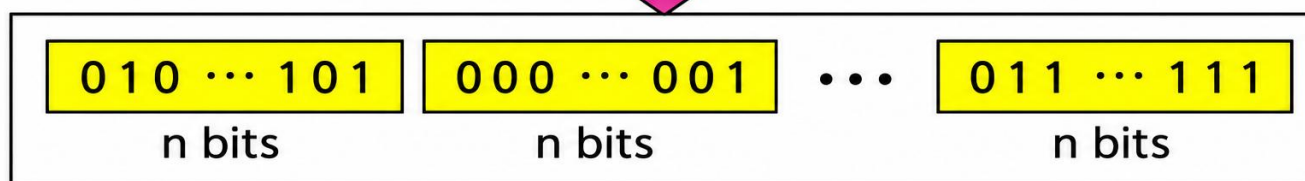
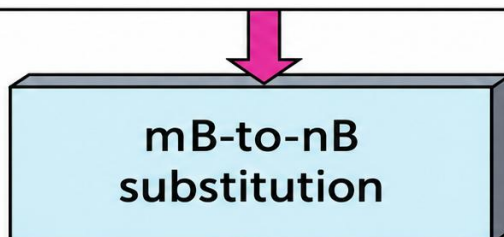
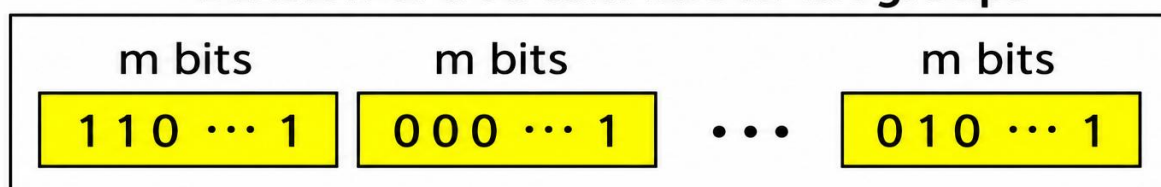
Data Sequence	Encoded Sequence	Control Sequence	Encoded Sequence
0000	11110	Q (Quiet)	00000
0001	01001	I (Idle)	11111
0010	10100	H (Halt)	00100
0011	10101	J (Start delimiter)	11000
0100	01010	K (Start delimiter)	10001
0101	01011	T (End delimiter)	01101
0110	01110	S (Set)	11001
0111	01111	R (Reset)	00111
1000	10010		
1001	10011		
1010	10130		
1011	10111		
1100	11010		
1101	11021		
1110	11100		
1111	11101		

The lookup table, which defines how each 4-bit group is converted into a 5-bit code.

Illustrates the process:

Divide the data → Convert each 4-bit group using the lookup table → Combine the resulting 5-bit codes and transmit them.

Division of a stream into m-bit groups



Combining n-bit groups into a stream

NOTE:

Unipolar → Simple, but it has several drawbacks.

RZ (Return-to-Zero) → Introduced to solve some of these problems.

Manchester and Differential Manchester → Improved **synchronization** between the transmitter and receiver.

Bipolar (AMI and Pseudoternary) → Eliminated the **DC component** problem.

B8ZS and HDB3 → Solved the problem of **long sequences of zeros**.

Block Coding (4B/5B and 8B/10B) → Modified the data before transmission to avoid all of these problems.

“THE END”

